

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 03 · LECTURE NOTES

Learning Values from Experience

Temporal difference, and the on/off-policy distinction.

MODULE OUTCOMES

- Derive the temporal-difference update.
- Compare Monte Carlo and temporal-difference learning.
- Distinguish on-policy from off-policy and demonstrate the difference.
- Explain n-step returns and eligibility traces.
- Explain maximisation bias and double learning.

1 Temporal-difference learning

$$V(s) \leftarrow V(s) + \alpha [r + \gamma V(s') - V(s)]$$

Move the estimate toward the observed reward plus the discounted estimate of the state you landed in. The bracketed quantity is the TD error — the discrepancy between your current prediction and a prediction that incorporates one step of real information — so **learning is driven by surprise**, and an accurate predictor stops updating. **It learns from a single transition without waiting for the episode to end**, which is what makes it usable online, in continuing tasks with no terminal state, and in episodes too long to wait out. **And it updates a guess toward another guess, which sounds unsound and nevertheless converges** (tabularly, with a decaying step size) — **because the second guess contains a real observed reward, so genuine information enters the system at every step and propagates backwards.**

	Monte Carlo	Temporal difference
Target	The actual return accumulated to the end of the episode.	$r + \gamma V(s')$ — bootstrapped from the current estimate.
Bias	Unbiased — it is a sample of the true return.	Biased while V is wrong, and the bias shrinks as V improves.
Variance	High — the target contains every random event in the rest of the episode.	Low — the target contains one step's randomness.
Requires	Episodes that terminate.	Nothing. Works online, incrementally, and in continuing tasks.
The Markov property	Does not rely on it — the return is the return.	Relies on it, because $V(s')$ is only a valid summary of the future if the state is sufficient.

It is a bias-variance trade (CSCE 633 Module 03 §2) **and n-step returns interpolate between the two endpoints** (§3) — **with the best n almost always strictly between 1 and the episode length**, which is the practical consequence and is why pure TD and pure Monte Carlo are both suboptimal defaults.

2 On-policy and off-policy

SARSA (on-policy):

```
Q(s,a) <- Q(s,a) + a[ r + g*Q(s',a') - Q(s,a) ]
where a' is the action ACTUALLY TAKEN next
```

Q-LEARNING (off-policy):

```
Q(s,a) <- Q(s,a) + a[ r + g*max_a' Q(s',a') - Q(s,a) ]
where the max is over ALL available actions
```

SARSA learns the value of THE POLICY IT IS FOLLOWING, exploration included. Q-learning learns the value of THE OPTIMAL POLICY, regardless of what it is actually doing.

THE CLIFF-WALKING DEMONSTRATION:

a grid world with a cliff along the bottom row. The shortest path runs right along the cliff edge. The agent behaves epsilon-greedily throughout.

Q-LEARNING learns the cliff-edge path -- correctly, since it IS optimal for a greedy policy -- and then FALLS OFF THE CLIFF REGULARLY, because epsilon occasionally steps sideways.

SARSA learns a path one row further from the cliff, which is SUBOPTIMAL for a greedy policy and BETTER for the epsilon-greedy policy it is actually running, because that policy sometimes steps randomly and SARSA has accounted for it.

SARSA ACHIEVES HIGHER ONLINE REWARD. Q-learning ends with the better final greedy policy.

THIS IS THE CLEAREST DEMONSTRATION OF THE DISTINCTION THAT EXISTS. Reproduce it once and it is permanent.

Off-policy learning is what allows experience replay and offline datasets (Modules 04 and 09) — if you can only learn from the policy you are currently running, you cannot reuse old experience at all, and sample efficiency becomes hopeless. **And it is also where instability comes from** (Module 04 §1's deadly triad names off-policy learning as one of its three ingredients), **which is the trade the rest of the course negotiates.**

3 Credit assignment over time

N-STEP RETURNS, AND WHY ELIGIBILITY TRACES EXIST

An n -step return uses n real observed rewards and then bootstraps from the estimate n steps ahead — so $n = 1$ is temporal difference and n equal to the episode length is Monte Carlo, and intermediate n interpolates §1's trade continuously.

The best n is almost always strictly between the endpoints, and $n = 3$ to 5 is a common and effective default. It is the single cheapest improvement available to a value-based agent — a few lines of buffering, no new hyperparameter beyond n , and a reliable gain.

Eligibility traces ($TD(\lambda)$) average over all n simultaneously, with geometrically decaying weights, implemented backwards by maintaining a decaying eligibility value per state — so the computational cost is one pass per step rather than n , which is the elegant part.

And traces are less used in deep reinforcement learning than they deserve to be, partly because replay buffers make the backward view awkward (the eligibility belongs to a trajectory and the buffer samples transitions independently) and partly because n -step returns capture most of the benefit with far less machinery. Which is an engineering accident rather than a result, and worth knowing in case your setting makes traces natural again.

4 Maximisation bias

THE MAX OF NOISY ESTIMATES IS BIASED UPWARD

Q-learning's target contains $\max_a Q(s', a)$, and the maximum of several noisy estimates exceeds the maximum of their true values in expectation — because the max selects whichever estimate happened to be most favourably perturbed. The effect requires no bias in the individual estimates at all; noise alone produces it.

So the bias is *systematic* rather than noise, and it **propagates**: an overestimated value becomes the bootstrap target for its predecessors, which overestimate in turn. **The error does not average out; it accumulates along trajectories.**

Double Q-learning fixes it by maintaining two estimators and using one to *select* the maximising action and the other to *evaluate* it — so the noise that chose the action is independent of the noise that values it, and the upward bias disappears.

And the same fix becomes Double DQN (Module 04 §3), where it is one of the very few modifications in deep reinforcement learning with an unambiguous, measurable, and well-understood benefit — which is worth saying because most of the field's reported improvements are neither unambiguous nor well-understood (Module 12).

- Q-learning with n -step returns and double estimation is the sensible tabular default, and it is what Module 04 scales up.

- **SARSA when online performance *during* learning matters** — a physical robot that must not fall off the cliff, or a deployed system whose exploration costs real money. §2's **demonstration is exactly this situation**.
- **Monte Carlo when episodes are short and the Markov property is doubtful**, since it does not rely on the state being a sufficient summary (§1's table) — which makes it surprisingly robust in partially observable settings.
- **And expected SARSA** — average over the policy's action distribution rather than sampling a' — **which has strictly lower variance than SARSA at no extra cost when the action set is small**, and is underused.
- **Everything in this module converges, tabularly, with proofs. Module 04 is where that stops being true** — so **implement all of these tabularly first**, because every deep method is one of them with a network in place of the table, and **the bugs are far easier to find when the algorithm is the only thing that can be wrong**.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sutton & Barto — chapters 5–7 and 12 (free PDF)	Monte Carlo, temporal difference, n-step, and traces — the reference for this whole module. The cliff-walking example of §2 is figure 6.5.
Watkins & Dayan — Q-learning (free)	The original convergence result — short, and worth reading for what the proof requires.
Hasselt — Double Q-learning (free)	§4's analysis and fix , with the bias demonstrated on a small example.
David Silver — Reinforcement Learning lectures 4–5 (free video)	Model-free prediction and control , and the clearest spoken explanation of §2's distinction.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement TD(0) and Monte Carlo** on a random walk and compare the error against episodes, averaged over runs.
- 2 **Measure the bias and variance of each target** empirically and confirm §1's table.
- 3 **Implement SARSA and Q-learning** on cliff walking and reproduce the two different paths.
- 4 **Plot online reward for both** and confirm SARSA is higher.
- 5 **Then plot the final greedy policy's performance** and confirm Q-learning is higher.

- 6 **Sweep n in n-step returns** from 1 to 20 and plot performance. Report the best n.
- 7 **Implement TD(λ)** with eligibility traces and compare against the best n-step.
- 8 **Construct a maximisation-bias example** and measure the overestimate.
- 9 **Implement double Q-learning** on it and confirm the bias disappears.
- 10 **Implement expected SARSA** and compare its variance against SARSA's.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the TD update and explain what the TD error is.
- 2 Why does updating a guess toward a guess converge?
- 3 Compare Monte Carlo and TD on five axes.
- 4 Which relies on the Markov property, and why?
- 5 Give the SARSA and Q-learning updates and state the difference precisely.
- 6 Describe cliff walking and what each algorithm learns.
- 7 What does off-policy learning enable, and what does it cost?
- 8 What do n-step returns interpolate, and what is a good default?
- 9 Explain maximisation bias and why it propagates.
- 10 How does double learning fix it?